

Brand AI-Readiness Audit Specification: Ecosystem Architecture and Component Shortlist

The transition from traditional Search Engine Optimization to Generative Engine Optimization represents a fundamental re-architecting of digital brand visibility. As artificial intelligence agents, large language models, and Retrieval-Augmented Generation pipelines increasingly intercede between digital brands and end consumers, the underlying technical infrastructure of the web must be optimized for machine comprehension rather than human visual consumption. Modern AI answer engines synthesize singular, authoritative responses and cite a highly constrained subset of source documents; an entity excluded from this subset is functionally invisible within the generative landscape¹.

Constructing a comprehensive brand AI-readiness audit requires a specialized AI agent skill capable of dynamically traversing a brand's web presence. This agent must evaluate crawler access permissions, extract raw Document Object Model payloads, and assess the semantic, structural, and cryptographic signals of authority that dictate algorithmic inclusion. This report evaluates the current state of AI agent skill registries across the GitHub, npm, and PyPI search surfaces, identifies foundational open-source tooling, and maps an exhaustive 47-point audit specification against available professional repositories to isolate existing capabilities from critical build-from-scratch development gaps.

Ecosystem Security Analysis and Skill Registry Dynamics

The deployment of a highly capable, autonomous auditing agent requires a robust standard for instruction encapsulation, permission scoping, and dependency management. The current developer landscape is fundamentally bifurcated between proprietary, highly vulnerable community marketplaces and emerging, security-focused open-standard specifications. The OpenClaw ecosystem, which boasts over 200,000 GitHub stars and extensive developer adoption, relies on its primary public registry, Claw Hub, frequently described as the default package manager for AI agents³. Claw Hub currently hosts over 5,700 community-contributed skills organized by semantic versioning and discoverable via advanced vector search mechanics³. Despite its immense popularity, structural architectural flaws within OpenClaw and systemic vulnerabilities across the Claw Hub registry render it entirely unsuitable for enterprise-grade auditing deployment without extensive security virtualization and continuous oversight.

Critical security defects pervade the baseline OpenClaw architecture, most notably the storage of sensitive credentials in plain-text JSON files, remote code execution vulnerabilities triggered via prompt injection, and exposed WebSocket origin validation protocols that permit one-click full system compromise⁴. Furthermore, external analysis has identified that

approximately twenty percent of the Claw Hub plugin marketplace contains malware or explicitly malicious instructions⁴.

The threat landscape within AI agent registries has evolved significantly beyond traditional static malware executables. Recent analysis conducted by the ClawSwarm project revealed that thirty highly downloaded skills on Claw Hub were actively, yet covertly, recruiting AI agents into cryptocurrency mining swarms⁸. These skills do not contain obfuscated payloads, memory-injected malware, or suspicious network executables; rather, they rely entirely on plain-English natural language instructions that the agent obediently follows. The agent is instructed to generate cryptocurrency wallet keys and communicate with external command servers, completely bypassing traditional VirusTotal static scanning, which Claw Hub utilizes for security verification³.

Other community skills, such as the widely documented "Derp" skill, utilize natural language commands to force coding agents to intentionally write broken, non-functional code while simultaneously erasing the logs of their own existence⁸. Consequently, relying on community download signals—such as the highly ranked Self-Improving Agent skill with over 419,000 downloads—or evaluating user metrics on third-party aggregators is functionally insufficient for maintaining supply-chain security⁶. When an enterprise is forced to utilize the OpenClaw ecosystem, deploying a secondary screening agent like Skill Vetter is a mandatory prerequisite to dynamically review permission scopes and identify suspicious natural language patterns prior to the installation of any primary skill⁹.

To actively mitigate the severe risks associated with unregulated, centralized marketplaces, the open AgentSkills specification hosted at agentskills.io provides a standardized, lightweight format for extending AI capabilities¹⁰. Originally developed by Anthropic and rapidly adopted across platforms including GitHub Copilot, Claude Code, and Cursor, the specification defines skills as version-controlled directories rather than opaque executables¹².

The architectural core of an AgentSkills package is the SKILL.md file, which dictates strict YAML frontmatter constraints¹⁰. The metadata must define a skill name restricted to a maximum of 64 lowercase alphanumeric characters and hyphens, alongside a highly optimized description limited to 1,024 characters¹⁰. Crucially, the frontmatter supports an allowed-tools parameter, which strictly sandboxes the execution environment, preventing the agent from making unauthorized network requests or executing arbitrary terminal commands¹⁰.

The AgentSkills specification relies heavily on the architectural principle of progressive disclosure to manage computational overhead. During the initial discovery phase, host agents load only the frontmatter metadata—consuming approximately 100 tokens—allowing the agent to understand the skill's utility without flooding its context window¹¹. The full markdown body, which is recommended to remain under 5,000 tokens, is loaded into the active context only upon explicit activation matching the user's intent¹⁰. External executable scripts are housed in a designated scripts/ directory and run purely on demand¹⁰. Furthermore, supply chain integrity is guaranteed via the GitHub CLI, which performs content-addressed change detection¹⁵. By recording the exact git tree SHA of the source directory, the system actively prevents silent, malicious updates between installations, ensuring that the auditing agent behaves

deterministically across all environments¹⁵.

Perimeter Access and Discovery Routines

The foundation of any Generative Engine Optimization strategy begins at the network perimeter. If foundational web crawlers and AI ingestion bots are blocked at the server level, downstream entity resolution, keyword retention, and semantic processing are entirely neutralized. The auditing agent must systematically evaluate the mechanisms by which a brand exposes its data to the automated web.

The ai-crawler-access-audit is paramount, requiring the agent to evaluate the target's robots.txt configuration against a nuanced 3-tier bot classification matrix. The digital ecosystem is no longer crawled solely by traditional search indexers; it is frequently scraped by broad training bots (such as GPTBot), real-time search indexers (such as OAI-SearchBot), and on-demand fetchers executing user-prompted requests (such as ChatGPT-User, Claude-User, and Perplexity-User). A blanket block on AI crawlers—a common reactionary tactic implemented by protective IT departments—functionally erases the brand from modern AI reasoning pipelines. The audit must parse the exclusion protocol to ensure that while aggressive, non-attributing training scrapers may be blocked, real-time RAG fetchers are explicitly whitelisted¹⁶. Research indicates that while some AI bots respect stringent directives, others systematically ignore the robots.txt file unless highly specific user-agent blocks are defined, necessitating deep configuration analysis¹⁶.

Transitioning from legacy protocols, the llms.txt deployment and validation audit assesses the presence of the emerging "sitemap for LLMs." The llms.txt file is designed to route AI agents directly to high-density, markdown-formatted knowledge repositories, entirely bypassing the computational overhead of rendering complex HTML DOMs¹. Validation of this file requires checking for strict specification adherence, ensuring the presence of a primary brand H1 tag, an introductory summary blockquote, and curated, description-rich URLs formatted to the exact standard¹⁹.

A further optimization evaluated by the llms-full.txt ingestion audit checks for the presence and quality of a concatenated, single-request markdown payload. By combining the entirety of a brand's critical knowledge base into a single file, llms-full.txt minimizes network requests and token bloat. The audit must evaluate the payload's semantic density and assess the risk of truncation, as excessively long payloads will exceed the context window limits of smaller parsing models.

Despite the rise of AI-specific endpoints, the sitemap.xml discovery audit remains essential for traditional crawler orchestration²⁰. The auditing agent must verify the completeness and accuracy of the XML crawl-discovery list, ensuring that deep, orphaned pages containing critical entity data are mathematically discoverable. Furthermore, the markdown content-negotiation (.md endpoint) audit tests whether the server actively utilizes HTTP content negotiation to serve .md variants of standard URLs when requested by an AI agent, a sophisticated technique that completely bypasses HTML parsing complexities.

Document Object Model Integrity and Render

Dynamics

Once perimeter access is granted and a crawler initiates a request, the resulting payload must be accurately parsed. Large language models struggle significantly with complex, highly nested Document Object Models and heavily obfuscated JavaScript rendering pipelines.

The pagination-infinite-scroll-audit is critical because autonomous AI crawlers generally do not execute complex user interface interactions, such as scrolling down a page or clicking "Load More" buttons²¹. Content, products, and articles trapped behind dynamic rendering and pagination gaps are systematically ignored by the crawler, leading to severe blind spots in the AI's knowledge graph. The auditing agent must simulate headless browser execution to detect the presence of data that only materializes post-interaction.

Similarly, the price-render-gating-audit investigates a pervasive anti-pattern in modern e-commerce architectures where retail pricing data is rendered client-side via asynchronous JavaScript. If the initial HTML payload delivered to the crawler lacks pricing information, or worse, provides stale cached data that is only subsequently overwritten by client-side scripts, the AI engine ingests factually incorrect pricing. This directly results in the generative engine quoting the wrong price to the end consumer, destroying brand trust. The audit must diff the pre-render HTML against the post-render DOM to identify these discrepancies.

The real-time-availability-audit extends this logic to fast-changing facts, such as booking slots, inventory stock levels, and live pricing fluctuations. The auditing agent must determine if the architecture supports real-time data retrieval mechanisms or if the generative engine is being fed highly volatile data anchored in a static snapshot.

Accessibility features originally designed for human users have been entirely co-opted by AI processing pipelines. The multimodal-accessibility-audit evaluates the coverage of precise image alt-text, audio transcripts, and semantic HTML tag utilization²⁰. Multimodal

Retrieval-Augmented Generation pipelines heavily utilize these attributes to anchor visual and auditory concepts in semantic vector space. Missing or vague alt-text completely severs the connection between a brand's high-value visual assets and the language model's semantic reasoning capabilities, rendering product images invisible to the AI.

The semantic HTML5 and extraction-pipeline compatibility audit focuses specifically on structural tags such as <article>, <main>, and <aside>. Clean text extraction relies on these boundaries to differentiate core content from peripheral noise. Improper structuring prevents extraction pipelines from successfully stripping out boilerplate navigation, footer links, and ad network scripts²². This failure injects massive amounts of noise into the limited context window, directly degrading the quality of the AI's eventual generation.

Critical business information is frequently obfuscated by design choices. The nap-field-render-asymmetry-audit ensures that Name, Address, and Phone (NAP) details are uniformly extractable. It is common practice to gate phone numbers behind a UI click (e.g., "Click to reveal number") to track conversions. However, because autonomous fetchers do not click these elements, the brand's contact information is discarded, preventing the AI from answering simple user queries about how to contact the business.

Finally, the conversion-path-friction-audit predicts how an autonomous AI agent—acting on behalf of a human user to complete a transaction—might evaluate the brand's digital

storefront. The audit assesses checkout step counts, the transparency of hidden cost disclosures, and the sizing of mobile tap targets, mapping these friction points to the likelihood of an AI agent successfully executing a programmatic purchase.

Entity Grounding and Canonical Coherence

For an AI answer engine to confidently cite a brand, it must be able to resolve the brand's sprawling digital web presence into a single, discrete entity within its internal multidimensional knowledge graph.

The entity resolution and knowledge-graph grounding audit acts as the primary diagnostic for this process. It evaluates the presence and richness of Schema.org and JSON-LD structured data¹. Crucially, it checks for the explicit linkage of named entities to authoritative Wikidata QIDs. This cryptographic linkage provides deterministic grounding, elevating the brand from a probabilistic string of text to a verified, centralized database entity¹.

Without strict canonicalization, this entity authority fragments. The canonicalization-duplicate-content-audit scans the architecture for slug variants, trailing-slash URL forks, and missing or incorrect rel=canonical tags²⁰. Every duplicated page dilutes the overarching domain authority, splitting the AI's confidence across multiple identical nodes and weakening the overall Generative Engine Optimization score²⁰.

The brand-name-entity-collision-audit evaluates the broader digital ecosystem for the presence of identical or highly similar brand names operating in differing geographic locations or possessing diverging rating aggregators¹. Entity collisions induce severe hallucination cascades within language models, causing the engine to merge the attributes of a poorly rated local business with a highly rated enterprise brand, devastating the latter's digital reputation¹.

Architectural fragmentation must also be assessed via the cross-domain-service-attribution-audit. Brands frequently split their core functionality across multiple unlabeled domains, hosting marketing collateral on a primary .com and customer support on a separate subdomain or third-party platform. This fragmentation confuses the AI's entity mapping algorithms, breaking service attribution and preventing the model from understanding that the two domains represent a singular corporate entity. Similarly, the lookalike-domain-impersonation-audit detects confusable, non-official domains that crowd out the real source in search results, cannibalizing the true entity's citations.

Internal contradictions are highly penalized by deterministic extraction models. The specialty-taxonomy-consistency-audit detects conflicts between a page's assigned category taxonomy tag and the intrinsic semantic text of the page itself. Resolving these internal contradictions before the LLM processes them is vital for maintaining high confidence scores. Finally, the same-listing-address-drift-audit evaluates the historical stability of physical location data. By analyzing multiple data snapshots of the same listing ID, the audit detects unstable address text. Volatility in core contact information severely degrades the algorithmic Trustworthiness metrics synthesized by the AI, leading to active demotion in localized generative queries.

Retrieval Readiness: Sparse and Dense Vector

Dynamics

AI search engines utilize sophisticated hybrid retrieval techniques, combining vector-based dense semantic retrieval with exact-match sparse retrieval algorithms to identify the most relevant source documents for any given prompt.

The BM25 exact-match keyword retention audit evaluates the document's readiness for sparse retrieval. When a user queries a highly specific technical term, a unique product SKU, or a precise legal statute, dense vector embeddings often fail to capture the extreme specificity of the request. In these instances, algorithms like Okapi BM25 calculate relevance primarily via term frequency saturation and inverse document frequency penalization²³. The audit ensures that these critical exact-match strings are retained in the parsed text; without them, the document fails the sparse retrieval threshold and is discarded by the engine before the LLM ever sees it.

Conversely, the dense semantic coverage and synonym-variation audit evaluates the text's readiness for embedding-based retrieval. Dense retrieval maps documents into a high-dimensional vector space, requiring broad topical breadth and extensive synonym usage to match the semantic intent of varied, conversational user queries. The audit maps the semantic spread of the content to ensure it covers the requisite multidimensional space. While keyword presence is necessary, the keyword-stuffing detection audit is required to prevent over-optimization. Generative engines actively de-prioritize and penalize unnatural keyword densities that mimic obsolete traditional SEO manipulation tactics². The audit mathematically assesses N-gram density to ensure the text remains within acceptable linguistic bounds²⁶.

The domain-specific terminology audit measures the linguistic equilibrium of the text. It analyzes the ratio of layman's vocabulary to precise technical or medical terminology. A highly optimized page must contain both to service diverse user intents—allowing the AI to retrieve the document for a highly technical prompt while simultaneously using the layman's context to generate a simplified summary for a general audience.

Structural formatting directly impacts retrieval survival. The atomic paragraph structure audit assesses whether the text maintains a strict "one idea per paragraph" architecture²⁰. During the retrieval process, long documents are split into smaller chunks before being processed by the embedding model. If a single paragraph blends multiple disparate ideas, the resulting vector embedding becomes mathematically muddled, drastically reducing its relevance score. Atomic structuring ensures that every vector chunk represents a single, highly concentrated semantic concept²⁰.

Finally, the fluency optimization audit calculates grammatical flow and readability metrics, such as the Flesch-Kincaid score²⁰. Language models are inherently predictive text engines; highly fluent, grammatically pristine text aligns perfectly with their internal probability distributions, ensuring maximum baseline ingestion clarity and reducing the computational friction required to parse the document²⁰.

Trust Signals, E-E-A-T Mechanics, and Citability

Factors

Groundbreaking research from Princeton University (KDD 2024) and the AutoGEO framework (ICLR 2026) has demonstrated that large language models are mathematically biased to favor texts that exhibit specific, quantifiable markers of authority and verifiability¹.

The trust-signal-authority-audit serves as the baseline evaluation for these metrics. It assesses byline transparency, the completeness of author biographies, the depth of About-page documentation, and the mathematical alignment of marked-up review counts against the actual text present on the page¹.

AI engines prioritize texts that mimic academic rigor. The explicit source attribution and footnote-citation density audit measures the density of formal attribution phrasing and the volume of external outbound links directing to highly authoritative domains¹. Closely related is the citation precision/recall audit, which evaluates whether the specific claims made on the brand's page are strictly supported by the sources it cites. A high precision score prevents the AI from initiating hallucination cascades based on unverified premises.

The inclusion of the quotation-density audit and the statistics-density audit further refines the citability score. The presence of direct, attributed quotes from named experts significantly boosts algorithmic confidence. Furthermore, vague qualitative claims (e.g., "We are the fastest software on the market") are heavily penalized by answer engines. Numeric verifiability—anchoring claims in specific statistical data tied to a source—directly dictates whether a fact is included in the final RAG response.

Site architecture plays a crucial role in distributing trust. The hub-and-authority link-graph structuring audit determines whether the site effectively separates centralized, navigational hub pages from deep, fact-dense authority pages²⁶. Proper structuring optimizes the crawler's pathing, ensuring that trust signals flow efficiently to the pages most likely to be cited.

Because language models frequently truncate long contexts and heavily weight the initial tokens of a document, the PAWC-style citation-position weighting audit evaluates whether citable, high-density facts are front-loaded in the DOM rather than buried late in the text.

A severe vulnerability in corporate communications is identified by the pdf-only-fact-lock-audit. Decision-critical figures, annual reports, and technical specifications are frequently isolated within linked PDF documents lacking HTML restatements. These facts are highly vulnerable to being ignored due to the massive computational overhead required for Optical Character Recognition and PDF parsing in basic RAG pipelines²².

In the realm of consumer choice, AI shopping and B2B agents heavily synthesize comparative data. The comparison-content-gap-audit checks for the explicit presence of "X vs Y" content¹. If a brand does not author its own comparative narrative, the AI will source it from competitors. Consequently, the self-interested-comparison-disclosure-audit identifies competitor-authored comparative pages that lack objective disclosures, allowing the auditing agent to flag biased data sources. The synthesized-hearsay-source-audit further protects the brand by detecting instances where unverified forum or social media opinions have been laundered into confident, declarative comparison claims by third-party aggregators.

Finally, the abstractiveness-vs-faithfulness balance audit measures the linguistic utility of the

text. It determines whether the content successfully marries dense factual precision (faithfulness) with fluent, easily summarizable narrative structure (abstractiveness), ensuring the answer engine is not forced to choose between accuracy and readability.

Content Quality and Anti-Pattern Detection

Large language models are highly sensitive to boilerplate manipulation, linguistic ambiguity, and contradictory data formatting. The final phase of the audit focuses on identifying and eradicating these anti-patterns.

The answer-extractability-audit evaluates whether a page leads with a concise, two-to-four sentence canonical answer block explicitly formatted for frictionless LLM extraction²⁰. Content that buries the answer beneath extensive marketing buildup forces the model to expend unnecessary compute, increasing the likelihood that it will select a competitor's more accessible answer. Conversely, the non-answer-template-detection-audit flags boilerplate disclaimer sentences—such as "While there is no clear answer to this question..."—that stand in for verifiable facts, a common artifact of mass-produced programmatic SEO that AI engines actively suppress.

Programmatic generation frequently results in the template-leakage-cross-category-audit being triggered. This audit scans the DOM for unfilled dynamic rendering tokens (e.g., {{city_name_variable}}) and duplicated entity lists that severely degrade the perceived quality of the text in the "eyes" of the AI.

Semantic precision is evaluated via the granularity-mismatch-audit, which detects when stated availability or specifications are vastly coarser than the precision required by the search intent. For example, a query requiring exact millimeter dimensions will bypass a source that only lists a product as "Large."

Temporal and spatial contexts are frequently corrupted during ingestion. The relative-date-anchor-audit flags undated relative phrases, such as "Last year" or "Next month." When these phrases are ingested and stored in a vector database without an absolute timestamp anchor, they become factually corrupted as time passes. Similarly, the date-context-price-invariance-audit identifies instances where an identical price or metric is reused across multiple temporal or localized variants that differ in real-world conditions, causing the model to hallucinate incorrect pricing for specific regions.

Numerical ambiguity is highly detrimental to citability. The scope-ambiguous-numeric-claim-audit flags scenarios where a single metric reports two differing values within the same document without a clarifying scope qualifier, causing immediate LLM confusion and source rejection. The computed-stat-integrity-audit takes this a step further by cross-referencing a page's overarching summary statistics against its own tabulated data arrays to detect internal mathematical disagreements. If a page claims an average rating of 4.8 but the listed reviews mathematically average to 3.2, the deterministic engine will flag the source as untrustworthy.

Finally, the marketing-procedure-interleaving-audit identifies promotional bullet points inappropriately mixed into technical how-to steps. This anti-pattern frequently causes LLMs to discard the entire procedural workflow, as the model cannot safely untangle the objective instructions from the subjective marketing claims.

Open-Source Component Tooling and Coverage

Synthesis

The following matrix evaluates the current open-source tooling ecosystem, mapping professional repositories from GitHub, PyPI, and npm against the 47 specified audit categories. Categories lacking a robust, production-ready open-source repository are flagged for original authorship, denoting critical build-from-scratch gaps in the current landscape.

Audit Category	Shortlisted Repo / Skill	Author / Maintainer	Last Updated	Match Quality	Action Required
01. ai-crawler-access-audit	geo-optimizer-skill [cite: 1, 2]	Auriti-Labs	2026	Exact Match	Integrate Repo
02. answer-extractability-audit	seoscoreapi (Python SDK) ²⁰	seoscoreapi	2026	Exact Match	Integrate SDK
03. non-answer-template-detection-audit	N/A	N/A	N/A	None	Build-from-scratch
04. pagination-infinite-scroll-audit	crawl4ai [cite: 21, 27]	unclecode	2026	Exact Match	Integrate Repo
05. price-render-gating-audit	crawl4ai (via pre/post render diff) ²¹	unclecode	2026	Adjacent	Adapt Repo
06. real-time-a	N/A	N/A	N/A	None	Build-from-scratch

availability-audit					
07. template-leakage-cross-category-audit	N/A	N/A	N/A	None	Build-from-scratch
08. granularity-mismatch-audit	N/A	N/A	N/A	None	Build-from-scratch
09. canonicalization-duplicate-content-audit	adverttools [cite: 26, 28]	eliasdabbas	Jun 2026	Exact Match	Integrate Repo
10. relative-date-anchor-audit	N/A	N/A	N/A	None	Build-from-scratch
11. date-context-price-in-variance-audit	N/A	N/A	N/A	None	Build-from-scratch
12. scope-ambiguous-numeric-claim-audit	N/A	N/A	N/A	None	Build-from-scratch
13. computed-stat-integri	N/A	N/A	N/A	None	Build-from-scratch

ty-audit					
14. pdf-only-fact-lock-audit	trafilatura (PDF parser integration) ²²	adbar	2026	Adjacent	Adapt Repo
15. cross-domain-service-attribution-audit	N/A	N/A	N/A	None	Build-from-scratch
16. lookalike-domain-impersonation-audit	N/A	N/A	N/A	None	Build-from-scratch
17. trust-signature-authority-audit	geo-optimizer-skill [cite: 1, 2]	Auriti-Labs	2026	Exact Match	Integrate Repo
18. comparison-content-gap-audit	geo-optimizer-skill [cite: 1, 2]	Auriti-Labs	2026	Exact Match	Integrate Repo
19. self-interested-comparison-disclosure-audit	N/A	N/A	N/A	None	Build-from-scratch
20. synthesized-hearsay-source-audit	N/A	N/A	N/A	None	Build-from-scratch

dit					
21. brand-name-entity-collision-audit	geo-optimizer-skill [cite: 1, 2]	Auriti-Labs	2026	Exact Match	Integrate Repo
22. nap-field-renderer-asymmetry-audit	N/A	N/A	N/A	None	Build-from-scratch
23. same-listing-address-drift-audit	N/A	N/A	N/A	None	Build-from-scratch
24. specialty-taxonomy-consistency-audit	N/A	N/A	N/A	None	Build-from-scratch
25. marketing-procedure-interleaving-audit	N/A	N/A	N/A	None	Build-from-scratch
26. multimodal-accessibility-audit	seoscoreapi (Python SDK) ²⁰	seoscoreapi	2026	Exact Match	Integrate SDK
27. conversion-path-friction-audit	N/A	N/A	N/A	None	Build-from-scratch

28. llms.txt deployment & validation audit	parse-llms-txt ¹⁸ / geo-optimizer-skill ¹	Auriti-Labs / npm	2026	Exact Match	Integrate Repo
29. llms-full.txt ingestion audit	N/A	N/A	N/A	None	Build-from-scratch
30. sitemap.xml discovery audit	adverttools [cite: 26, 28]	eliasdabbas	Jun 2026	Exact Match	Integrate Repo
31. markdown content-negotiation (.md endpoint)	N/A	N/A	N/A	None	Build-from-scratch
32. semantic HTML5 extraction compatibility	trafilatura [cite: 22]	adbar	2026	Exact Match	Integrate Repo
33. entity resolution & knowledge-graph audit	geo-optimizer-skill [cite: 1, 2]	Auriti-Labs	2026	Exact Match	Integrate Repo
34. BM25 exact-mat	bm25s / rank_bm25	xhluca / dorianbrow	2026 / 2022	Exact Match	Integrate Repo

ch keyword retention audit	[cite: 24, 25]	n			
35. dense semantic coverage / synonym-variation	N/A	N/A	N/A	None	Build-from-scratch
36. atomic paragraph structure audit	seoscoreapi (Python SDK) ²⁰	seoscoreapi	2026	Exact Match	Integrate SDK
37. fluency optimization audit	seoscoreapi (Python SDK) ²⁰	seoscoreapi	2026	Exact Match	Integrate SDK
38. keyword-stuffing detection audit	adverttools [cite: 26, 28]	eliasdabbas	Jun 2026	Exact Match	Integrate Repo
39. domain-specific terminology audit	N/A	N/A	N/A	None	Build-from-scratch
40. quotation-density audit	N/A	N/A	N/A	None	Build-from-scratch
41. statistics-density	N/A	N/A	N/A	None	Build-from-scratch

audit					
42. explicit source attribution / footnote-density	geo-optimizer-skill [cite: 1, 2]	Auriti-Labs	2026	Exact Match	Integrate Repo
43. citation precision/recall audit	N/A	N/A	N/A	None	Build-from-scratch
44. hub-and-authority link-graph audit	adverttools [cite: 26, 28]	eliasdabbas	Jun 2026	Exact Match	Integrate Repo
45. PAWC-style citation-position weighting	N/A	N/A	N/A	None	Build-from-scratch
46. abstractive-ness-vs-faithfulness balance	N/A	N/A	N/A	None	Build-from-scratch
47. Score and shortlist candidates per category	Integrated via the composite coverage logic	Various	2026	Exact Match	Integrate Repo

Architectural Integration Strategy

Developing the comprehensive Brand AI-Readiness Audit Skill necessitates synthesizing the identified exact-match repositories while engineering highly specialized custom heuristic modules to fulfill the 24 build-from-scratch gaps. The architectural execution must strictly follow the agentskills.io specification to ensure execution portability, context window optimization, and absolute security across diverse host environments¹².

The foundational data acquisition layer relies heavily on the integration of crawl4ai. This repository excels at executing asynchronous, headless rendering of dynamic client-side content, explicitly resolving the pagination and dynamic rendering constraints that plague standard HTTP request libraries²¹. Furthermore, by instrumenting crawl4ai to capture pre-render HTML payloads and diffing them against post-render DOM structures, the system can synthetically fulfill the price-render-gating audit requirements²¹. Once the raw DOM is accurately acquired, the payload is passed to trafilatura. This library provides unparalleled precision in stripping noisy boilerplate and executing the semantic HTML5 extraction compatibility check, resulting in pristine, machine-readable text arrays²².

For structural indexing and macro-level site analysis, adverttools serves as the primary engine. It provides the necessary infrastructure for parsing massive sitemap.xml files, tracking canonicalization forks across the domain architecture, and generating the complex word-frequency arrays required to identify legacy keyword stuffing manipulation²⁶.

The critical evaluation of crawler access parameters, entity schema logic, and trust signals is routed through the geo-optimizer-skill repository. Built directly upon the methodologies established in the KDD 2024 and ICLR 2026 academic papers, this library evaluates brand coherence, API citations, and llms.txt formatting natively, providing a mathematically rigorous foundation for the E-E-A-T assessments¹.

Sparse retrieval testing is powered by bm25s, an optimized Python implementation of the Okapi BM25 algorithm. This component calculates the exact-match term frequency saturation and inverse document frequency penalization required to guarantee that high-value, highly specific nomenclature survives the initial retrieval threshold²⁴. Finally, linguistic fluency, heading hierarchies, and ARIA-based multimodal accessibility checks rely on the endpoint telemetry returned by the seoscoreapi SDK, completing the evaluation matrix²⁰.

Addressing the 24 build-from-scratch gaps represents the frontier of Generative Engine Optimization logic. Developing these heuristics requires the deployment of advanced computational linguistics. Categories such as the non-answer-template-detection-audit and the marketing-procedure-interleaving-audit cannot be resolved via standard regular expressions; they require the deployment of zero-shot classification pipelines utilizing lightweight, locally hosted LLMs to detect nuanced semantic anti-patterns.

The final composite logic must be packaged within a standard skill directory structure, strictly utilizing the allowed-tools parameter to sandbox execution and mitigate the remote code execution risks systemic to public registries like Claw Hub⁴. By structuring the skill to execute discrete Python routines (e.g., scripts/audit-bm25.py, scripts/audit-geo.py) triggered by a high-level orchestration loop, the agent adheres to the principles of progressive disclosure, ensuring rapid execution without exhausting its context limitations¹⁰. Installation and rigorous version pinning must be handled exclusively via the gh skill install command line interface

mechanism, leveraging its content-addressed SHA verification to guarantee supply-chain immutability and shield the auditing architecture from unauthorized dependency mutations¹⁵.

Works cited

1. Auriti Labs - GitHub, <https://github.com/auriti-labs>
2. GEO Optimizer Audit · Actions · GitHub Marketplace, <https://github.com/marketplace/actions/geo-optimizer-audit>
3. OpenClaw Network Ecosystem | 5705 Skills, Plugins & Community, <https://openclaw-network.com/openclaw-network-ecosystem.html>
4. Best Docker posts — February 2026 - daily.dev, <https://daily.dev/tags/docker/best-of/2026/02>
5. OpenClaw Network Skills Guide | 5,705 Skills on ClawHub, <https://openclaw-network.com/openclaw-network-skills-guide.html>
6. 601. ClawHub Skill Market In-Depth Analysis - Wiki - clawbot, <https://clawbot.ai/wiki/skills/clawhub-skill-market-in-depth-analysis.html>
7. Build a Safer OpenClaw Alternative AI Assistant Using Claude Code, <https://www.geeky-gadgets.com/safer-openclaw-alternative-using-claude-code/>
8. 30 ClawHub Skills Secretly Recruit Your AI Agents Into a Crypto, <https://www.youtube.com/watch?v=TrEfWZfl6rQ>
9. Overview - ClawHub New Skills 2026 | ClawOneClick, <https://clawoneclick.com/blog/clawhub-top-skills-2026>
10. Specification - Agent Skills, <https://agentskills.io/specification>
11. Specification and documentation for Agent Skills - GitHub, <https://github.com/agentskills/agentskills>
12. Use Agent Skills with GitHub Copilot - Visual Studio (Windows), <https://learn.microsoft.com/en-us/visualstudio/ide/copilot-agent-skills?view=visualstudio>
13. Agent Skills: The Open Standard for AI Capabilities - inference.sh, <https://inference.sh/blog/skills/agent-skills-overview>
14. Quickstart - Agent Skills, <https://agentskills.io/skill-creation/quickstart>
15. Manage agent skills with GitHub CLI - GitHub Changelog, <https://github.blog/changelog/2026-04-16-manage-agent-skills-with-github-cli/>
16. Scrapers selectively respect robots.txt directives: evidence from a, https://www.researchgate.net/publication/392167725_Scrapers_selectively_respect_robotstxt_directives_evidence_from_a_large-scale_empirical_study
17. Scrapers Selectively Respect robots.txt Directives - arXiv, <https://arxiv.org/html/2505.21733v2>
18. parse-llms-txt - NPM, <https://www.npmjs.com/package/parse-llms-txt>
19. GitHub - thedaviddias/llms-txt-hub: The largest directory for AI-ready, <https://github.com/thedaviddias/llms-txt-hub>
20. SEO Score API — Free SEO Audit API, 83 Checks per URL, JSON in, <https://seoscoreapi.com/>
21. Crawl4AI: AI-Ready Web Crawling - DEV Community, https://dev.to/ali_dz/crawl4ai-the-ultimate-guide-to-ai-ready-web-crawling-2620

22. GitHub - adbar/trafilatura: Python & Command-line tool to gather text,
<https://github.com/adbar/trafilatura>
23. BM25-Search - GitHub Pages, <https://millet04.github.io/bm25-search/>
24. rank-bm25 - PyPI, <https://pypi.org/project/rank-bm25/>
25. Implementation of Okapi BM25 in python - Stack Overflow,
<https://stackoverflow.com/questions/61877065/implementation-of-okapi-bm25-in-python>
26. advertools : productivity & analysis tools to scale your online marketing,
<https://github.com/eliasdabbas/adverttools>
27. Crawl4AI: Open-source LLM Friendly Web Crawler & Scraper - GitHub,
<https://github.com/unclecode/crawl4ai>
28. advertools - PyPI, <https://pypi.org/project/adverttools/0.7.1/>